

* Reinforcement Learning.

↳ exploitation vs exploration

↳ Policy: mapping from perceived states to action

↳ reward signal: mapping from state x actions $\rightarrow$ reward

↳ Value function: total reward hoped to be accumulated in the future starting from the state.

↳ rewards are immediate, value is cumulative over the future.

↳ model: models the environment: mapping from state x actions $\rightarrow$ state

* Temporal Difference learning:

↳ values are assigned to all states $\in [0, 1]$

↳ known dead or win states are given 0, 1 : rest are given 0.5

↳ greedy policy usually taken

↳ exploratory steps $\rightarrow$ not greedy / randomly chosen } $\rightarrow$ only update for greedy policy

$$V(s) \leftarrow V(s) + \alpha (V(s') - V(s))$$

s' is the next step state

↳ Coding implementation of Tic Tac Toe

temporal difference learning:

↳ Learning: generate all possible moves $\rightarrow$ evaluate & initialise values for future states

↳ update values based on the state after opponent move

↳ $s, a \rightarrow s' \leftarrow$ player move $\rightarrow$ if game ended $V(s) \leftarrow V(s) + \alpha (V(s') - V(s))$

↳ $s', a' \rightarrow s'' \leftarrow$ opponent move $\rightarrow V(s) \leftarrow V(s) + \alpha (V(s'') - V(s))$

↳ apply only for greedy moves

↳ exploratory moves: just choose one of the legal moves

↳ parameters: $\alpha \leftarrow$ step size $\approx$ learning rate

$p \leftarrow$ exploration rate.

↳ $V(s'') - V(s) =$ temporal difference

* N-Bandit Problem:

↳ n choices, each with rewards from fixed probability distributions

↳ find how to maximise the reward.

↳ Action-Value methods:

↳ an action a has reward expectation true $= q(a)$

↳ predicted or estimated reward at timestep $t = Q_t(a) = \frac{\sum R_i}{N_t(a)} \leftarrow$ avg of all rewards received when action "a" was taken till timestep t

as $N_t(a) \rightarrow \infty$ $Q_t(a) \rightarrow q(a)$

↳ greedy choice $= A_t^* = \arg \max_a [Q_t(a)] \leftarrow$ most of the time but also explore $\rightarrow \epsilon$ greedy approach

↳ exploration important.

↳ keep Rolling track of average: $Q_{k+1} = \frac{k Q_k + R_k}{k+1} = \frac{(k+1) Q_k + R_k - Q_k}{k+1} : Q_k + \frac{1}{k+1} (R_k - Q_k)$

: of the form New estimate $\leftarrow$ Old Estimate + StepSize $\times$ (Target - Old Estimate)

↳ works for stationary distributions

↳ $Q_{k+1} = Q_1 (1-\alpha)^k + \sum_{i=1}^k \alpha (1-\alpha)^{k-i} R_i$ } weighted average + shift

↳ for convergence

$\sum \alpha = \infty$ $\sum \alpha^2 < \infty$ works for $1/k$ but not for fixed α

since it allows action value to change

Stuff thats changing:

No.:	Ⓐ α	Ⓑ ϵ	Ⓒ Q_1	Page No.
	↳ more important if non static distribution ↳ if $\propto 1/K$ then good for static distributions	↳ more exploration ↳ if too much $\Rightarrow$ reward drops	↳ more exploration initially, dies down after a while ↳ may be slow initially but better in long run ↳ encodes initially known info abt the distribution	

*UCB: upper confidence Bound:

↳ kinda adds a buffer for uncertainty

↳ instead of using ϵ for exploration, includes it into value of the action

$$A_t = \operatorname{argmax}_a (Q_t(a) + \epsilon \sqrt{\frac{\ln(t)}{N_t(a)}})$$

↳ $\ln(t)$ ensures exploration slows down with time

↳ $\propto 1/N_t(a)$ will give more chance to unexplored options

↳ ϵ is still better cuz unfeasible to keep track of N_t for high no. of states

↳ not suitable for changing distributions $\leftarrow$ gives imp to history: may overlook changes

*Gradient Bandits:

↳ associates preferences instead of probabilities

↳ probabilities are calculated in response to these preferences

$$\pi_t(a) = \frac{e^{H_t(a)}}{\sum_i e^{H_t(a_i)}}$$

The preference is updated after every action

let A_t be the action taken

$$H_{t+1}(A_t) = H_t(A_t) + \alpha (R_t - \bar{R}_t) (1 - \pi_t(A_t))$$

stochastic version of

$$H_{t+1} = H_t + \frac{\partial E(R_t)}{\partial H_t} \alpha$$

check if the taken choice was better than usual $\leftarrow$ reduces effect if it was already favoured

$H_{t+1}(a) = H_t(a) + \alpha (R_t - \bar{R}_t) \pi_t(a)$ $\leftarrow$ reduces the preference of all other actions

$$\frac{\partial E(R_t)}{\partial H_t(a)} = \frac{\partial}{\partial H_t(a)} \left[\sum_b \pi_t(b) q(b) \right] = \sum_b q(b) \frac{\partial \pi_t(b)}{\partial H_t(a)}$$

x_t stabilizes the rewards, helps notice small differences

mathematically equivalent

$$\text{when expanded } \alpha \sum_b \frac{\partial \pi_t(b)}{\partial H_t(a)} \Rightarrow \sum_b (q(b) - x_t) \frac{\partial \pi_t(b)}{\partial H_t(a)}$$

$$\Rightarrow \sum_b \pi(b) \frac{(q(b) - x_t) \frac{\partial \pi_t(b)}{\partial H_t(a)}}{\pi(b)} = E \left[\frac{(q(b) - \bar{R}_t) \frac{\partial \pi_t(b)}{\partial H_t(a)}}{\pi(b)} \right] = \frac{q(b) \neq \bar{R}_t}{\text{but } E(q(b)) = E(\bar{R}_t)}$$

$$= E \left[\frac{R_t - \bar{R}_t}{\pi(A_t)} \pi(A_t) (\mathbb{I}_{a=A_t} - \pi_t(a)) \right] \quad \frac{\partial \pi_t(b)}{\partial H_t(a)} = \frac{\partial}{\partial H_t(a)} \frac{e^{H_t(b)}}{\sum_c e^{H_t(c)}} = \frac{\partial e^{H_t(b)}}{\partial H_t(a)} \frac{1}{\sum_c e^{H_t(c)}} - \frac{e^{H_t(b)}}{\sum_c e^{H_t(c)}} \frac{\partial \sum_c e^{H_t(c)}}{\partial H_t(a)}$$

$$= E [R_t - \bar{R}_t (\mathbb{I}_{a=A_t} - \pi_t(a))]$$

↳ do this at every step & its an approximation of gradient ascent

$$\left(\sum_c e^{H_t(c)} \right)^2$$

$$H_{t+1}(a) = H_t(a) + \alpha (R_t - \bar{R}_t) (\mathbb{I}_{a=A_t} - \pi_t(a))$$

$$= \frac{e^{H_t(b)} \mathbb{I}_{b=a}}{\sum_c e^{H_t(c)}} - \frac{e^{H_t(b)} e^{H_t(a)}}{\left(\sum_c e^{H_t(c)} \right)^2}$$

same as before.

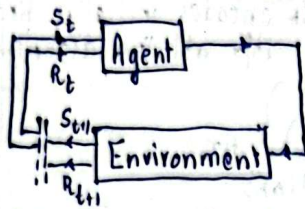
$$= \mathbb{I}_{b=a} \pi_t(b) - \pi_t(b) \pi_t(a) = \pi_t(b) (\mathbb{I}_{b=a} - \pi_t(a))$$

Effect on α : if α is too large then overshooting & oscillations
 ↳ if normal gradient ascent problem
 ↳ adaptive to baseline.

* Associative Search:

↳ associate policy actions & rewards to some set of situations

* Finite Markov Decision Process:



$s_t \in S \Rightarrow \text{Policy } \pi_t$
 $a_t \in A(s_t)$
 $R_t \in R$
 $\pi_t(a|s) = \text{prob of taking action 'a' given current state is 's'}$

↳ expected return at timestep $t = G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} \dots + R_T$ till termination of episode
 ↳ goal is to maximize expected return at each timestep.

↳ Tasks → Episodic: $G_t = R_{t+1} + R_{t+2} + R_{t+3} \dots R_T \rightarrow \text{maximise +ve rewards}$

↳ S ← set of non-terminal states; S^+ ← set of all states

↳ Continual: $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$ $0 \leq \gamma \leq 1$

↳ $\gamma = 0$: myopic

↳ $\gamma = 1$: far-sighted

↳ minimise or delay penalties

↳ unifying both notations: $G_t = \sum_{k=0}^{T-t} \gamma^k R_{t+k+1}$

episodic $\Rightarrow \gamma = 1 \Rightarrow T \neq \infty$ or if $T = \infty$
 continuous $\Rightarrow \gamma < 1 \Rightarrow T = \infty$
 ↳ terminal absorbing state is introduced

↳ Markov Property: only depends on prev state & actions, not history

$$Pr\{R_{t+1}=r, S_{t+1}=s' | S_t, A_t\} = Pr\{R_{t+1}=r, S_{t+1}=s' | S_t, A_t, R_t, S_{t-1}, A_{t-1}, R_{t-1}, \dots, S_0, A_0, R_0\}$$

history independent

↳ Markov Decision Processes:

$$P(s', r | s, a) = Pr\{S_{t+1}=s', R_{t+1}=r | S_t=s, A_t=a\}$$

$$p(r | s, a) = \sum_{s' \in S} P(s', r | s, a) \quad \hookrightarrow p(s' | s, a) = \sum_{r \in R} P(s', r | s, a) = P_{ss'}^a$$

$$r(s, a) = E[R_{t+1} | S_t=s, A_t=a] = \sum_{r \in R} r P(r | s, a)$$

$$\begin{aligned} \hookrightarrow r(s, a, s') &= E[R_{t+1} | S_{t+1}=s', S_t=s, A_t=a] = \sum_{r \in R} r P(s', r | s, a) P(r | s, a) \\ &= \sum_{r \in R} r \frac{P(s', r | s, a)}{P(s' | s, a)} \end{aligned}$$

↳ recycle Bot MDP problem:

S	S'	a	$P(s' s, a)$	$r(s, a)$
H	H	S	α	r_{search}
H	L	S	$1-\alpha$	r_{search}
H	H	W	0	r_{wait}
H	L	W	0	r_{wait}
L	L	S	β	r_{search}
L	H	S	$1-\beta$	-3 ← rescued
L	L	W	0	r_{wait}
L	H	W	0	r_{wait}
L	H	R	1	0
L	L	R	0	0

↳ Value functions:

$$V_{\pi}(s) = E[G_t | S_t=s] = E_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t=s \right]$$

$$q_{\pi}(s, a) = E[G_t | S_t=s, A_t=a]$$

$$V_{\pi}(s) = \sum_a \pi(a|s) \sum_{s', r} P(s', r | s, a) (r + \gamma V_{\pi}(s'))$$

* Bellman Equation

$$V_{\pi}(s) = \sum_{a \in A(s)} \pi(a|s) q_{\pi}(s, a)$$

* Optimal Value Functions: $V_*(s) = \max_{\pi} V_{\pi}(s) \forall s \in S$

No.:

$$q_*(s, a) = \max_{\pi} q_{\pi}(s, a)$$

Page No.

$$q_*(s, a) = E[R_{t+1} + \gamma V_*(S_{t+1}) | S_t = s, A_t = a]$$

$$V_*(s) = \max_a q_*(s, a) = \max_a E[R_{t+1} + \gamma V_*(S_{t+1}) | S_t = s, A_t = a]$$

$$= \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V_*(s')]$$

$$q_*(s, a) = E[R_{t+1} + \gamma \max_{a'} q_*(s', a') | S_t = s; a_t = a] = \sum_{s', r} p(s', r | s, a) [r + \gamma \max_{a'} q_*(s', a')]$$

∴ Bellman Optimality Equations & Intuition.

$$V_*(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V_*(s')] \leftarrow \text{the optimal policy will always take the greedy approach to maximise } q_*(s, a)$$

For that action take $\sum_{s', r} p(s', r | s, a) [\dots]$ ← sum of rewards for that being the next option, this should be the optimal V_*

$$q_*(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma \max_{a'} q_*(s', a')] \leftarrow \text{take current action but then behave optimally this gives the optimal value for the given state action pair.}$$

Eg. Solving Bellman Optimality for recycle bot: $V_*(h)$

$$V_* = \max_a \left\{ \sum_{s', r} p(s', r | s, a) [r + \gamma V_*(s')] \right\} = \max \left\{ \begin{aligned} & p(h, r_s | h, s) [r_s + \gamma V_*(h)] \\ & + p(l, r_s | h, s) [r_s + \gamma V_*(l)] \\ & + p(h, r_w | h, w) [r_w + \gamma V_*(h)] \\ & + p(l, r_w | h, w) [r_w + \gamma V_*(l)] \end{aligned} \right\}$$

$$V_*(h) = \max \left\{ \begin{aligned} & \alpha(r_s + \gamma V_*(h)) + (1-\alpha)(r_s + \gamma V_*(l)) \\ & 1x(r_w + \gamma V_*(h)) + 0x(\dots) \end{aligned} \right\} \rightarrow \textcircled{1} p(l, r_w | h, w) [r_w + \gamma V_*(l)]$$

$$V_*(l) = \max \left\{ \begin{aligned} & p(l | l, s) (\beta(r_s + \gamma V_*(l)) + (1-\beta)(-3 + \gamma V_*(h))) \\ & p(l | l, w) (r_w + \gamma V_*(l)) + p(h | l, w) (r_w + \gamma V_*(h)) \\ & p(l | l, r) (0 + \gamma V_*(h)) + p(h | l, r) (0 + \gamma V_*(h)) \end{aligned} \right\}$$

$$= \max \left\{ \begin{aligned} & \beta(r_s + \gamma V_*(l)) + (1-\beta)(-3 + \gamma V_*(h)) \\ & r_w + \gamma V_*(l) \\ & \gamma V_*(h) \end{aligned} \right\} \rightarrow \textcircled{2}$$

For $\alpha, \beta, \gamma, r_s, r_w$ there is a unique solution when $\textcircled{1}$ & $\textcircled{2}$ are solved simultaneously

Ex 3.12: $V_{\pi}(s) = \sum_{\pi(a|s)} \pi(a|s) q_{\pi}(s, a) = E[G_t | S_t = s] = E[\sum_{\pi} q_{\pi}(s, a) | S_t = s]$

Ex 3.13: $q_{\pi}(s, a) = E[R_{t+1} + \gamma V_{\pi}(S_{t+1}) | S_t = s, A_t = a] = \sum_{s', r} p(s', r | s, a) (r + \gamma V_{\pi}(s'))$

Ex 3.16: state action pairs: $(h, s) (h, w) (l, s) (l, w) (l, r)$

$$q_*(h, s) = \sum_{s', r} p(s', r | h, s) (r + \gamma \max_{a'} q_*(s', a')) = \max \left\{ \begin{aligned} & \alpha(r_s + \gamma \max_{a'} q_*(h, a')) \\ & + (1-\alpha)(r_s + \gamma \max_{a'} q_*(l, a')) \\ & \alpha(r_s + \gamma V_*(h)) + (1-\alpha)(r_s + \gamma V_*(l)) \end{aligned} \right\}$$

Ex 3.18: $V_*(s) = \max_a q_*(s, a)$

Ex 3.19: $q_*(s, a) = E[R_{t+1} + \gamma V_*(s'_{t+1}) | s_t = s, a_t = a]$
 $= \sum_{s', r} p(s', r | s, a) (r + \gamma V_*(s'))$

Ex 3.20: $\pi_*(a|s) = 1$ if $q_*(s, a) = \max_{a'} q_*(s, a')$
 0 otherwise

Ex 3.21: $\pi_*(a|s) = 1$ for 'a' for which $\sum_{s', r} p(s', r | s, a) (r + \gamma V_*(s'))$ is max
 0 otherwise

* Dynamic Programming:

↳ Iterative Policy evaluation: learn V iteratively $V_{k+1}(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) (r + \gamma V_k(s'))$

↳ Policy evaluation: ↳ Policy Improvement:

$\pi'(s) = \operatorname{argmax}_a q_\pi(s, a) = \operatorname{argmax}_a E[R_{t+1} + \gamma V_\pi(s') | s_t = s, a_t = a]$

this ensures that $\pi'(s) \geq \pi(s)$ $\pi'(s) = \pi(s) \forall s \Rightarrow \pi = \pi^*$

↳ Policy Iteration: $\pi_0 \xrightarrow{E} V_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} V_{\pi_1} \xrightarrow{I} \pi_2 \dots \xrightarrow{E} V_{\pi^*} \xrightarrow{I} \pi^*$

For the given Gridworld problem:

Policy Optimization after evaluating once:

0	←	←	↙
↑	↖	↖	↓
↑	↖	↗	↓
↙	→	→	0

Optimal Policy

* Jack Trade DP: had stochastic element

↳ remember to always compute

$E[R_{t+1} + \gamma V_k(s')]$

↳ all can be initialised to 0

* Gambler's problem: initialize the final value = 1

↳ kinda backpropagate based on that

* Asynchronous MDP: sweeps can be taken on latest data arbitrarily.

* General Policy Iteration: $\pi \xrightarrow{\text{evaluation}} V \xrightarrow{\text{Improvement}} \pi$

* Value Iteration:

update only wrt most strong action

$V_{k+1}(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V_k(s')] \leftarrow$ only one iteration for policy evaluation

still converges.

↳ entire concept of DP needs knowledge about the model i.e. $p(s', r | s, a)$
 this might not be available.

* Monte Carlo Methods: → Basic Algorithms:

↳ learn from experience.

↳ only for episodic tasks

↳ independent estimates for each states

↳ Eg. Black Jack Problem: policy found.

*** Try solving using DP

$\pi \leftarrow$ policy to be evaluated

$V \leftarrow$ empty

Returns [s] $\leftarrow$ 0 initialised list

Generate episodes over policy π

for each state calculate G_t after first occurrence

Append $G_t(s)$ to Returns [s]

$V(s) \leftarrow \text{average}(\text{return}[s])$

$$G_t = \underbrace{\gamma(1-\gamma)R_{t+1}}_{\substack{\text{ends after 1} \\ \text{round}}} + \underbrace{\gamma(1-\gamma)(R_{t+1}+R_{t+2})}_{\substack{\text{doesn't} \\ \text{end in } R_{t+2}}} + \gamma^2(1-\gamma)(R_{t+1}+R_{t+2}+R_{t+3}) \dots \gamma^{T-t-1}(R_{t+1}+R_{t+2}+\dots+R_{T-2})$$

$$= \gamma^{T-t-1}(1-\gamma)G_t + \sum_{h=t+1}^{T-1} (1-\gamma)\gamma^{h-t-1}\bar{G}_t^h$$

Same formula for G_t can be used in simple & weighted Importance sampling

$$V(s) = \frac{\sum_{t \in \tau(s)} \left(\gamma^{T-t-1} \bar{G}_t^T \rho_t^{\tau(t)} + \sum_{h=t+1}^{T-1} (1-\gamma)\gamma^{h-t-1} \bar{G}_t^h \rho_t^{\tau(t)} \right)}{\sum_{t \in \tau(s)} \left(\gamma^{T-t-1} \rho_t^{\tau(t)} + \sum_{h=t+1}^{T-1} (1-\gamma)\gamma^{h-t-1} \rho_t^{\tau(t)} \right)} \quad \left. \begin{array}{l} \text{weighted} \\ \leftarrow |V(s)| \text{ for standard} \end{array} \right\}$$

* Temporal Difference Learning:

↳ estimates $V_\pi(s)$ and optimizes

↳ bootstrapping from DP and sampling from Monte Carlo

$$V_\pi(s) = E_\pi \left(R_{t+1} + \gamma V_\pi(s') \mid S_t = s \right)$$

↳ algorithm: run thru episode:

take action A according to policy $\xrightarrow{\text{observe}} R, S'$

update $V(s) \leftarrow V(s) + \alpha (R + \gamma V(s') - V(s))$ } temporal diff
estimate of what $V(s)$ should be

↳ implemented in a fully incremental fashion

↳ more about closely modelling the actual problem rather than just reducing error

↳ TD control on the state action pairs rather than state values

$$Q(s, A) \leftarrow Q(s, A) + \alpha [R_{t+1} + Q(s', A') - Q(s, A)]$$

S' is state after A, A' chosen from $\pi(S')$

↳ This is on policy $\rightarrow$ SARSA algorithm since it depends on

$(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}) \leftarrow$ jumping between 2 state action pairs

↳ OFF policy TD learning = Q learning $\leftarrow Q(s_t, A_t) \leftarrow Q(s_t, A_t) + \alpha (R + \max_a Q(s_{t+1}, a) - Q(s_t, A_t))$

↳ also converges to Q values

↳ Afterstate: state after taking action but before environment has responded

↳ better due to efficiency $\leftarrow$ multiple (s, a) have the same afterstate.

↳ better to estimate afterstate value function.

↳ slightly modified: expected SARSA: $Q(s_t, A_t) \leftarrow Q(s_t, A_t) + \alpha (R_{t+1} + \gamma \sum_a \pi(a|s_{t+1}) Q(s_{t+1}, a) - Q(s_t, A_t))$

↳ generally converges faster than normal SARSA
↳ on-policy

* Eligibility Traces:

↳ how many rewards / steps are considered for re-evaluation of V or Q

↳ bridge between Monte Carlo and TD learning

↑ episode backup ↑ 1 step backup

↳ Eg. for n -step Backup: $G_t^{t+n} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(s_{t+n})$

$$V_{t+1}(s) = V_t(s) + \alpha (G_t^{t+n} - V_t(s))$$

* n-step TD is inconvenient to implement.

* Complex backups: take weight average of multiple

No.: * TD(λ) ← way of performing such a weighted average across diff number of episodes.

$$G_t = (1-\lambda) \sum_{k=0}^{T-t-1} \lambda^k G_{t+k} + \lambda^{T-t-1} G_T \quad \text{Forward View}$$

* $\lambda = 1 \rightarrow$ Monte Carlo

for $\lambda = 0 \rightarrow$ TD(0) is simple TD

* Backward View of TD(λ): how to computationally accomplish.

↳ have an additional variable for all states called eligibility trace

$$E_t(s) \rightarrow \text{updated as follows: } \rightarrow E_{t+1}(s) = \lambda \gamma E_t(s) \text{ if } s \neq s_t$$

$$= \lambda \gamma E_t(s) + 1$$

↳ Algorithm:

Initialize all traces $\{E\}$

↳ Repeat: $A \leftarrow \text{action} \rightarrow \text{observe } R, S'$

$\delta \leftarrow R + \gamma V(S') - V(S)$

For episodes $E(S) = E(S) + 1$ or 1 or $(1-\alpha)E(S) + 1$

For $s \in S: V(s) \leftarrow V(s) + \alpha \delta E(s); E(s) \leftarrow \gamma \lambda E(s)$

↳ Forward & Backward views are equivalent when done offline

↳ There is a "True" Online TD(λ)

$$V_{t+1}(s) = V_t(s) + \alpha [\delta_t + V_t(s_t) - V_{t-1}(s_t)] E_t(s) - \alpha I_{ss_t} [V_t(s_t) - V_{t-1}(s_t)]$$

↳ Watkin's $\alpha(\lambda)$: instead of using eligibility traces till the end of the episode

↳ used till the next exploratory move

↳ after that the $E(s)$ resets

↳ Off policy eligibility traces become useless or too complicated

↳ maybe use importance sampling somewhere.

↳ computational boost: only update S for which $E(S) \in [0, \epsilon]$ some threshold

↳ Variable λ can have a time-step dependent λ

↳ λ can be a function of S or t $\lambda(S) \leftarrow$ helps decide if $V(S)$ is known or estimated correctly

* Planning and Learning: $\rightarrow$ Monte Carlo & Qlearning & temporal difference $\leftarrow$ learning

$\rightarrow$ Dynamic Programming & heuristic search $\leftarrow$ planning

* models predict how the environment responds i.e next state and reward

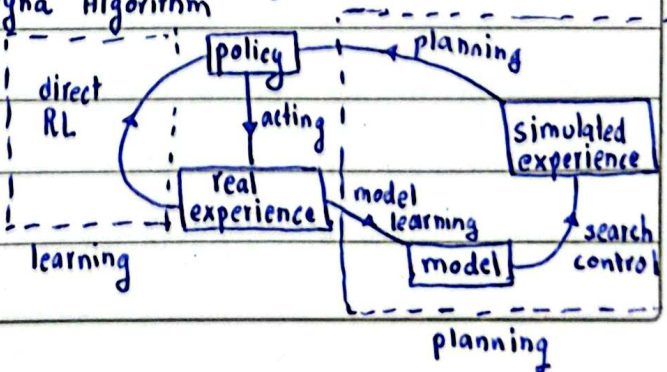
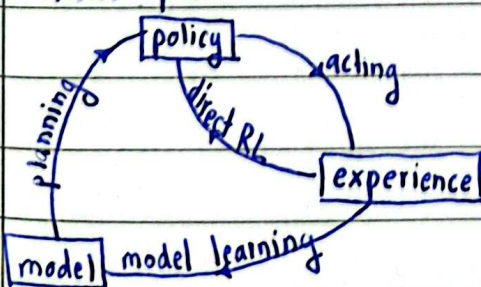
↳ distribution models: given outcome & its probability

↳ sample modes: given outcome with samples drawn according to distribution

* planning: takes a model and optimizes a policy according to it

* Master plan:

* Dyna Algorithm



* Dyna-Q algorithm: Dyna Version of Q learning:

model(S, A) $\rightarrow S', R' \leftarrow$ initialize; $Q(S, a) = 0 \forall S, a$

Do forever: $A \leftarrow \epsilon$ -Greedy(Q, S)

execute $A \rightarrow S', R$ $\max_a$

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', a) - Q(S, A)]$ } $\leftarrow$ direct RL

model(S, A) $\leftarrow R, S'$ } $\leftarrow$ model learning

Repeat for n episodes:

$S \leftarrow$ start state or randomly chosen state

$A \leftarrow$ seen action from S

$R, S' \leftarrow$ model(S, A)

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

} planning
from simulated
experiences

* What if the model is wrong / or / model / environment changes:

$\rightarrow$ if current model is more optimistic $\leftarrow$ will learn eventually

$\rightarrow$ if current model is pessimistic $\leftarrow$ might never learn if not for heuristic strategies

$\rightarrow$ Eg of model is optimistic $\rightarrow$ shortcut in maze is blocked

* Dyna-Q* model:

heuristic approach: keeps track of last time the (state, action) pair was tried

$\rightarrow$ intuition: the more time has elapsed the more time has elapsed $\leftarrow$ the more probable it is to have changed

$\rightarrow$ given an additional reward to resample old states

$\rightarrow R' = R + K \sqrt{t} \leftarrow$ time since it was last tried
coeff.

* Prioritized sweeps:

$\rightarrow$ instead of randomly choosing and backing up states

$\rightarrow$ add them (S, A) to a Pqueue if $|S| > \theta$: then backpropagate from that state

$\rightarrow$ do backup until $n_{\max}$ is reached or Pqueue is empty

* trajectory sample:

$\rightarrow$ simulate a trajectory

$\rightarrow$ better for larger problems

* Full Backups Vs Sample Backups:

$\rightarrow$ if multiple future states exist lets say b

$\rightarrow$ full backup comp

= single backup comp

$\rightarrow$ performance wise it may be useful to do a fraction of b backups to get approx results

* Heuristic Search:

$\rightarrow$ construct a tree to decide policy based on current value function

$\rightarrow$ extend branches deeper for branches that look promising

$\rightarrow$ take backup from this tree which has

$\rightarrow$ helps correct for approximation errors

*SUMMARY: Concepts & Formulae

No.:

① N-Bandit Problem:

Page No.

↳ no states, n-actions

↳ action value method:

$$\rightarrow Q_{k+1}(a) = Q_k + \frac{1}{k} (R_k - Q_k) \leftarrow \text{simple averaging}$$

$$\rightarrow Q_{k+1}(a) = Q_k(a) + \alpha (R_k - Q_k(a)) \leftarrow \text{fixed } \alpha$$

$$\rightarrow \text{policy: greedy} = \operatorname{argmax}_a (Q(a))$$

exploration: ϵ rate

$$\rightarrow \text{UCB} = A_T = \operatorname{argmax}_a \left(R Q_1(a) + \epsilon \sqrt{\frac{\ln(t)}{N_t(a)}} \right)$$

↳ alternative to ϵ

↳ Gradient Bandits:

↳ preferences to all actions: $H_t(a)$

$$\rightarrow \pi(a) = \frac{e^{H_t(a)}}{\sum_b e^{H_t(b)}}$$

↳ optimized using stochastic gradient ascent

$$H_{t+1}(a) = H_t(a) + (R_t - R_t) (\mathbb{I}_{a=A_t} - \pi(a))$$

② Markov Decision Process:

$$\rightarrow V_{k+1}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) (r + \gamma V_k(s'))$$

↳ Bellman Equation

$$\rightarrow V_{\pi}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) (r + \gamma V_{\pi}(s'))$$

$$\rightarrow V_*(s) = \max_a \sum_{s',r} p(s',r|s,a) (r + \gamma V_*(s'))$$

$$\rightarrow q_*(s,a) = \sum_{s',r} p(s',r|s,a) (r + \gamma \max_{a'} q_*(s',a'))$$

↳ Bellman Optimality Equation

③ Dynamic Programming:

↳ Policy Evaluation:

$$\rightarrow V_{k+1}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) (r + V_k(s'))$$

↳ Policy Improvement:

$$\rightarrow \pi'(s) = \operatorname{argmax}_a \left[\sum_{s',r} p(s',r|s,a) (r + V_{\pi}(s')) \right]$$

policy iteration:
done cyclicly
until optimal
policy
is reached

$$\rightarrow \text{Value Iteration: } V_{k+1}(s) = \max_a \sum_{s',r} p(s',r|s,a) (r + V_k(s'))$$

④ Monte-Carlo Methods:

$$\hookrightarrow V_{\pi}(s) = \frac{\sum G_t(s)}{N}$$

$\hookrightarrow$ off-policy: importance sampling:

$$\hookrightarrow \rho_t^{\pi^{(l)}} = \prod_{k=t}^{T^{(l)}-1} \frac{\pi(A_k|S_k)}{\mu(A_k|S_k)}$$

$$\hookrightarrow V(s) = \frac{\sum_{t \in \mathcal{T}(s)} \rho_t^{\pi^{(l)}} G_t(s)}{N_t(s) = |\mathcal{T}(s)|}$$

← simple averaging.

weighted imp sampling

$$\hookrightarrow V(s) = \frac{\sum_{t \in \mathcal{T}(s)} \rho_t^{\pi^{(l)}} G_t(s)}{\sum_{t \in \mathcal{T}(s)} \rho_t^{\pi^{(l)}}}$$

$\hookrightarrow$ Truncated episodes:

$$V(s) = \frac{\sum_{t \in \mathcal{T}(s)} \left(\gamma^{T^{(l)}-t-1} G_t^{\pi^{(l)}} \rho_t^{\pi^{(l)}} + (1-\gamma) \sum_{h=t+1}^{T^{(l)}-1} \gamma^{h-t-1} \bar{G}_t^h \rho_t^{\pi^{(l)}} \right)}{|\mathcal{T}(s)|}$$

$\hookrightarrow$ weighted IS

$$|\mathcal{T}(s)| = \sum_{t \in \mathcal{T}(s)} \left(\gamma^{T^{(l)}-t-1} \rho_t^{\pi^{(l)}} + (1-\gamma) \sum_{h=t+1}^{T^{(l)}-1} \gamma^{h-t-1} \rho_t^{\pi^{(l)}} \right)$$

for simple IS

⑤ Temporal Difference Learning:

$$\hookrightarrow V_{\pi}(s) = V_{\pi}(s) + \alpha (R + \gamma V_{\pi}(s') - V_{\pi}(s))$$

$$\hookrightarrow \text{SARSA: } Q(s,a) \leftarrow Q(s,a) + \alpha (R + \gamma Q(s',a') - Q(s,a))$$

$$\hookrightarrow Q \text{ learning: } Q(s,a) \leftarrow Q(s,a) + \alpha (R + \gamma \max_{a'} Q(s',a') - Q(s,a))$$

$$\hookrightarrow \text{expected SARSA: } Q(s,a) \leftarrow Q(s,a) + \alpha (R + \gamma \sum_{a' \in A(s')} \pi(a'|s') Q(s',a') - Q(s,a))$$

⑥ Eligibility Traces:

$\hookrightarrow \lambda$ = parameter

$$\hookrightarrow L_t = (1-\lambda) \sum_{k=1}^{T-t-1} \lambda^{k-1} G_t^{t+k} + \lambda^{T-t-1} G_t$$

$$\hookrightarrow E_t(s) \begin{cases} \rightarrow E_{t+1}(s) = E_t(s) + 1 \\ \rightarrow E_{t+1}(s) = (1-\alpha) E_t(s) + 1 \\ \rightarrow E_{tf}(s) = 1 \end{cases} \rightarrow E_{t+1}(s)^* = \gamma \lambda$$

$$\hookrightarrow V(s) = V(s) + \alpha \delta E(s) \text{ where } \delta = R + \gamma V(s') - V(s)$$

⑦ Planning & Learning: Dyna Q+ : $R + c\sqrt{e}$